AI Lab Experiments 2 to 8 (Python)

Exp 2: McCulloch-Pitts (AND, OR, XOR)

```
def mp(x, w, t):
    return int(sum(a * b for a, b in zip(x, w)) >= t)

for a in (0, 1):
    for b in (0, 1):
        print(a, b,
              "AND:", mp([a, b], [1, 1], 2),
              "OR:", mp([a, b], [1, 1], 1),
              "XOR:", mp([mp([a, b], [1, -1], 1),
                          mp([a, b], [-1, 1], 1)], [1, 1], 1))
```

Exp 3: Fuzzy operations

```
A = [0.2, 0.5, 0.8]
B = [0.6, 0.3, 0.9]

print("Union:", [max(a, b) for a, b in zip(A, B)])
print("Intersection:", [min(a, b) for a, b in zip(A, B)])
print("Complement:", [round(1 - a, 1) for a in A])
```

Exp 4: Fuzzy extension (single and two variable)

```
A = {-2: 0.3, -1: 0.6, 0: 1, 1: 0.7, 2: 0.4}
B = {1: 0.5, 2: 1, 3: 0.2}
Y = {}
Z = {}

for x in A:
    Y[x * x] = max(Y.get(x * x, 0), A[x])
    for y in B:
        Z[x + y] = max(Z.get(x + y, 0), min(A[x], B[y]))

print("Single variable y = x^2:", Y)
print("Two variables z = x + y:", Z)
```

Exp 5: Supervised NN (Hebbian AND, Perceptron OR)

```
X = [(1, 1), (1, -1), (-1, 1), (-1, -1)]
AND = [1, -1, -1, -1]
OR = [1, 1, 1, -1]

# Hebbian rule (AND)
w1 = w2 = b = 0
for (x1, x2), y in zip(X, AND):
    w1 += x1 * y
    w2 += x2 * y
    b += y
print("Hebbian:", w1, w2, b)

# Perceptron rule (OR)
w1 = w2 = b = 0
for epoch in range(1, 5):
    for (x1, x2), y in zip(X, OR):
        out = 1 if w1 * x1 + w2 * x2 + b >= 0 else -1
        if out != y:
            w1 += x1 * y
            w2 += x2 * y
            b += y
    print("Epoch", epoch, w1, w2, b)
```

Exp 6: Unsupervised NN (Autoencoder)

```
import numpy as np

X = np.array([[1, 0, 1, 0], [0, 1, 0, 1], [1, 1, 1, 1], [0, 0, 0, 0]])
W1 = np.random.rand(4, 2)    # encoder
W2 = np.random.rand(2, 4)    # decoder

for epoch in range(2000):
    H = X @ W1               # encode
    Y = H @ W2               # decode
    E = Y - X                # error
    W1 -= 0.05 * X.T @ (E @ W2.T)
    W2 -= 0.05 * H.T @ E

print(np.round(Y))
```

Exp 7: Fuzzy controller (washing machine)

```
import matplotlib.pyplot as plt

def tri(x, a, b, c):
    return max(min((x - a) / (b - a), (c - x) / (c - b)), 0)

dirt = 80
low = tri(dirt, -50, 0, 50)
med = tri(dirt, 0, 50, 100)
high = tri(dirt, 50, 100, 150)

time = (low * 10 + med * 20 + high * 30) / (low + med + high)
print("Wash time:", time)

plt.plot([-50, 0, 50], [0, 1, 0], [0, 50, 100], [0, 1, 0], [50, 100, 150], [0, 1, 0])
plt.show()
```

Exp 8: Genetic algorithm (maximize x^2)

```
import random

def f(x):
    return x * x

pop = [random.randint(0, 31) for _ in range(4)]

for g in range(10):
    # selection: best two win
    pop.sort(key=f, reverse=True)
    a, b = pop[0], pop[1]

    # crossover
    c = (a & 24) | (b & 7)
    d = (b & 24) | (a & 7)
    pop = [a, b, c, d]

    # mutation
    pop[3] ^= 1 << random.randint(0, 4)

    print(g, max(pop, key=f))
```